

What is Semi-Structured Data?

Semi-structured data is data that **does not follow a strict tabular schema like relational databases**, but still contains **organizational structure** such as tags or key-value pairs.

Unlike structured data, the schema is **flexible and can change over time**.

Common formats:

- JSON
- XML
- Avro
- Parquet
- ORC

Example JSON data

```
{
  "customer_id":101,
  "name":"Rohith",
  "city":"Bangalore",
  "orders":[
    {"order_id":1,"amount":200},
    {"order_id":2,"amount":350}
  ]
}
```

Here the **orders field contains nested data**, which makes it semi-structured.

Why Semi-Structured Data is Important

Modern applications generate large amounts of semi-structured data.

Examples:

- Web applications
- Mobile applications
- IoT devices
- API responses
- Log files

Traditional relational databases struggle to handle **dynamic schemas**, so modern warehouses like **Snowflake support semi-structured data natively**.

Semi-Structured Data Types in Snowflake

Snowflake provides special data types to store semi-structured data.

Data Type Description

VARIANT Stores JSON, XML, Avro, etc

OBJECT Stores key-value pairs

ARRAY Stores list of values

Example table:

```
CREATE TABLE customer_json
(  
  data VARIANT  
);
```

The **VARIANT** column can store complete JSON objects.

Data Loading Architecture

Typical semi-structured data pipeline:

Application Logs / APIs



Cloud Storage (S3 / Azure Blob)



Snowflake Stage



Raw Table (VARIANT column)



Flatten / Transform



Structured Tables



Analytics / BI

Example Real-Time Scenario

Industry: **E-commerce Platform**

API generates JSON order data.

Example JSON file:

```
{  
  "order_id":1001,  
  "customer_id":101,  
  "items":[  
    {"product":"Laptop","price":800},  
    {"product":"Mouse","price":20}  
  ],  
}
```

```
"order_date":"2024-01-01"  
}
```

This file is uploaded to **AWS S3**.

Snowflake loads this data for analytics.

Step 1 – Create Stage

First create a **stage to access cloud storage**.

Example:

```
CREATE STAGE order_stage  
URL='s3://company-orders-data/';
```

The stage connects Snowflake to the storage location.

Step 2 – Create Target Table

Create a table with **VARIANT column**.

```
CREATE TABLE raw_orders  
(  
  order_data VARIANT  
);
```

This table stores **complete JSON records**.

Step 3 – Load JSON Data

Use **COPY INTO command**.

Example:

```
COPY INTO raw_orders  
FROM @order_stage  
FILE_FORMAT = (TYPE = JSON);
```

Snowflake automatically parses the JSON file and loads it into the **VARIANT column**.

Viewing Loaded Data

Example query:

```
SELECT order_data  
FROM raw_orders;
```

Output example

order_data

```
{ "order_id":1001, "customer_id":101, ... }
```

Accessing JSON Fields

Snowflake allows accessing JSON fields using **dot notation**.

Example:

```
SELECT
order_data:order_id,
order_data:customer_id
FROM raw_orders;
```

Result:

order_id customer_id

```
1001    101
```

Casting Data Types

JSON values are stored as **VARIANT**, so we convert them to proper types.

Example:

```
SELECT
order_data:order_id::INTEGER,
order_data:order_date::DATE
FROM raw_orders;
```

Handling Nested Data (FLATTEN)

Semi-structured data often contains **arrays or nested objects**.

Example JSON

```
"items":[
{"product":"Laptop","price":800},
{"product":"Mouse","price":20}
]
```

To extract array values, Snowflake uses **FLATTEN function**.

Example:

```
SELECT
order_data:order_id,
item.value:product,
item.value:price
FROM raw_orders,
LATERAL FLATTEN(input => order_data:items) item;
```

Result

order_id	product	price
----------	---------	-------

1001	Laptop	800
------	--------	-----

1001	Mouse	20
------	-------	----

Transforming into Structured Tables

After flattening, data is inserted into structured tables.

Example table:

```
CREATE TABLE order_items
(
order_id INT,
product STRING,
price NUMBER
);
```

Insert transformed data:

```
INSERT INTO order_items
SELECT
order_data:order_id::INT,
item.value:product::STRING,
item.value:price::NUMBER
FROM raw_orders,
LATERAL FLATTEN(input => order_data:items) item;
```

Now the semi-structured data becomes **structured and query-ready**.

Advantages of Semi-Structured Data in Snowflake

- Flexible schema
 - Easy ingestion of JSON/XML
 - Native support for nested data
 - No schema redesign needed when fields change
-

Best Practices

For production pipelines:

- Store raw data in **VARIANT columns**
 - Perform transformations in **staging tables**
 - Use **FLATTEN for nested arrays**
 - Convert data types before loading to final tables
-

Common Interview Questions

What is semi-structured data?

Data that does not follow a fixed schema but still contains structured elements like **key-value pairs or tags**.

Which Snowflake data type stores semi-structured data?

VARIANT

Which function is used to extract nested arrays?

FLATTEN

What command loads JSON files into Snowflake?

COPY INTO

Quick Summary

Semi-structured data allows organizations to store **flexible data formats like JSON, XML, and logs**. Snowflake supports this data natively using **VARIANT columns**, enabling easy ingestion, transformation, and analytics on complex nested datasets.